Lucien Roberson
CS5600
HW3

Chapter 8:

Q1:

```
vboxuser@Lucien:~/5600/hw3$ python3 mlfq.py -s 1 -j 2 -n 2 -m 30 -M 0 -q 10 -c
Here is the list of inputs:
OPTIONS jobs 2
OPTIONS queues 2
OPTIONS allotments for queue  1 is   1
OPTIONS quantum length for queue  1 is  10
OPTIONS allotments for queue  0 is   1
OPTIONS quantum length for queue  0 is  10
OPTIONS boost 0
OPTIONS ioTime 5
OPTIONS stayAfterIO False
OPTIONS iobump False


For each job, three defining characteristics are given:
  startTime : at what time does the job enter the system
  runTime   : the total CPU time needed by the job to finish
  ioFreq    : every ioFreq time units, the job issues an I/O
              (the I/O takes ioTime units to complete)

Job List:
  Job  0: startTime   0 - runTime   4 - ioFreq   0
  Job  1: startTime   0 - runTime  23 - ioFreq   0
```

```
Execution Trace:

[ time 0 ] JOB BEGINS by JOB 0
[ time 0 ] JOB BEGINS by JOB 1
[ time 0 ] Run JOB 0 at PRIORITY 1 [ TICKS 9 ALLOT 1 TIME 3 (of 4) ]
[ time 1 ] Run JOB 0 at PRIORITY 1 [ TICKS 8 ALLOT 1 TIME 2 (of 4) ]
[ time 2 ] Run JOB 0 at PRIORITY 1 [ TICKS 7 ALLOT 1 TIME 1 (of 4) ]
[ time 3 ] Run JOB 0 at PRIORITY 1 [ TICKS 6 ALLOT 1 TIME 0 (of 4) ]
[ time 4 ] FINISHED JOB 0
[ time 4 ] Run JOB 1 at PRIORITY 1 [ TICKS 9 ALLOT 1 TIME 22 (of 23) ]
[ time 5 ] Run JOB 1 at PRIORITY 1 [ TICKS 8 ALLOT 1 TIME 21 (of 23) ]
[ time 6 ] Run JOB 1 at PRIORITY 1 [ TICKS 7 ALLOT 1 TIME 20 (of 23) ]
[ time 7 ] Run JOB 1 at PRIORITY 1 [ TICKS 6 ALLOT 1 TIME 19 (of 23) ]
[ time 8 ] Run JOB 1 at PRIORITY 1 [ TICKS 5 ALLOT 1 TIME 18 (of 23) ]
[ time 9 ] Run JOB 1 at PRIORITY 1 [ TICKS 4 ALLOT 1 TIME 17 (of 23) ]
[ time 10 ] Run JOB 1 at PRIORITY 1 [ TICKS 3 ALLOT 1 TIME 16 (of 23) ]
[ time 11 ] Run JOB 1 at PRIORITY 1 [ TICKS 2 ALLOT 1 TIME 15 (of 23) ]
[ time 12 ] Run JOB 1 at PRIORITY 1 [ TICKS 1 ALLOT 1 TIME 14 (of 23) ]
[ time 13 ] Run JOB 1 at PRIORITY 1 [ TICKS 0 ALLOT 1 TIME 13 (of 23) ]
[ time 14 ] Run JOB 1 at PRIORITY 0 [ TICKS 9 ALLOT 1 TIME 12 (of 23) ]
[ time 15 ] Run JOB 1 at PRIORITY 0 [ TICKS 8 ALLOT 1 TIME 11 (of 23) ]
[ time 16 ] Run JOB 1 at PRIORITY 0 [ TICKS 7 ALLOT 1 TIME 10 (of 23) ]
[ time 17 ] Run JOB 1 at PRIORITY 0 [ TICKS 6 ALLOT 1 TIME 9 (of 23) ]
[ time 18 ] Run JOB 1 at PRIORITY 0 [ TICKS 5 ALLOT 1 TIME 8 (of 23) ]
[ time 19 ] Run JOB 1 at PRIORITY 0 [ TICKS 4 ALLOT 1 TIME 7 (of 23) ]
[ time 20 ] Run JOB 1 at PRIORITY 0 [ TICKS 3 ALLOT 1 TIME 6 (of 23) ]
[ time 21 ] Run JOB 1 at PRIORITY 0 [ TICKS 2 ALLOT 1 TIME 5 (of 23) ]
[ time 22 ] Run JOB 1 at PRIORITY 0 [ TICKS 1 ALLOT 1 TIME 4 (of 23) ]
[ time 23 ] Run JOB 1 at PRIORITY 0 [ TICKS 0 ALLOT 1 TIME 3 (of 23) ]
[ time 24 ] Run JOB 1 at PRIORITY 0 [ TICKS 9 ALLOT 1 TIME 2 (of 23) ]
[ time 25 ] Run JOB 1 at PRIORITY 0 [ TICKS 8 ALLOT 1 TIME 1 (of 23) ]
[ time 26 ] Run JOB 1 at PRIORITY 0 [ TICKS 7 ALLOT 1 TIME 0 (of 23) ]
[ time 27 ] FINISHED JOB 1

Final statistics:
  Job  0: startTime   0 - response   0 - turnaround   4
  Job  1: startTime   0 - response   4 - turnaround  27

  Avg  1: startTime n/a - response 2.00 - turnaround 15.50
```

Q2)
Example 1:

```
vboxuser@Lucien:~/5600/hw3$ python3 ./mlfq.py -n 3 -q 10 -l 0,200,0 -c
Here is the list of inputs:
OPTIONS jobs 1
OPTIONS queues 3
OPTIONS allotments for queue  2 is   1
OPTIONS quantum length for queue  2 is  10
OPTIONS allotments for queue  1 is   1
OPTIONS quantum length for queue  1 is  10
OPTIONS allotments for queue  0 is   1
OPTIONS quantum length for queue  0 is  10
OPTIONS boost 0
OPTIONS ioTime 5
OPTIONS stayAfterIO False
OPTIONS iobump False


For each job, three defining characteristics are given:
  startTime : at what time does the job enter the system
  runTime   : the total CPU time needed by the job to finish
  ioFreq    : every ioFreq time units, the job issues an I/O
              (the I/O takes ioTime units to complete)

Job List:
  Job  0: startTime   0 - runTime 200 - ioFreq   0


Execution Trace:

[ time 0 ] JOB BEGINS by JOB 0
[ time 0 ] Run JOB 0 at PRIORITY 2 [ TICKS 9 ALLOT 1 TIME 199 (of 200) ]
[ time 1 ] Run JOB 0 at PRIORITY 2 [ TICKS 8 ALLOT 1 TIME 198 (of 200) ]
[ time 2 ] Run JOB 0 at PRIORITY 2 [ TICKS 7 ALLOT 1 TIME 197 (of 200) ]
[ time 3 ] Run JOB 0 at PRIORITY 2 [ TICKS 6 ALLOT 1 TIME 196 (of 200) ]
[ time 175 ] Run JOB 0 at PRIORITY 0 [ TICKS 4 ALLOT 1 TIME 24 (of 200) ]
[ time 176 ] Run JOB 0 at PRIORITY 0 [ TICKS 3 ALLOT 1 TIME 23 (of 200) ]
[ time 177 ] Run JOB 0 at PRIORITY 0 [ TICKS 2 ALLOT 1 TIME 22 (of 200) ]
[ time 178 ] Run JOB 0 at PRIORITY 0 [ TICKS 1 ALLOT 1 TIME 21 (of 200) ]
[ time 179 ] Run JOB 0 at PRIORITY 0 [ TICKS 0 ALLOT 1 TIME 20 (of 200) ]
[ time 180 ] Run JOB 0 at PRIORITY 0 [ TICKS 9 ALLOT 1 TIME 19 (of 200) ]
[ time 181 ] Run JOB 0 at PRIORITY 0 [ TICKS 8 ALLOT 1 TIME 18 (of 200) ]
[ time 182 ] Run JOB 0 at PRIORITY 0 [ TICKS 7 ALLOT 1 TIME 17 (of 200) ]
[ time 183 ] Run JOB 0 at PRIORITY 0 [ TICKS 6 ALLOT 1 TIME 16 (of 200) ]
[ time 184 ] Run JOB 0 at PRIORITY 0 [ TICKS 5 ALLOT 1 TIME 15 (of 200) ]
[ time 185 ] Run JOB 0 at PRIORITY 0 [ TICKS 4 ALLOT 1 TIME 14 (of 200) ]
[ time 186 ] Run JOB 0 at PRIORITY 0 [ TICKS 3 ALLOT 1 TIME 13 (of 200) ]
[ time 187 ] Run JOB 0 at PRIORITY 0 [ TICKS 2 ALLOT 1 TIME 12 (of 200) ]
[ time 188 ] Run JOB 0 at PRIORITY 0 [ TICKS 1 ALLOT 1 TIME 11 (of 200) ]
[ time 189 ] Run JOB 0 at PRIORITY 0 [ TICKS 0 ALLOT 1 TIME 10 (of 200) ]
[ time 190 ] Run JOB 0 at PRIORITY 0 [ TICKS 9 ALLOT 1 TIME 9 (of 200) ]
[ time 191 ] Run JOB 0 at PRIORITY 0 [ TICKS 8 ALLOT 1 TIME 8 (of 200) ]
[ time 192 ] Run JOB 0 at PRIORITY 0 [ TICKS 7 ALLOT 1 TIME 7 (of 200) ]
[ time 193 ] Run JOB 0 at PRIORITY 0 [ TICKS 6 ALLOT 1 TIME 6 (of 200) ]
[ time 194 ] Run JOB 0 at PRIORITY 0 [ TICKS 5 ALLOT 1 TIME 5 (of 200) ]
[ time 195 ] Run JOB 0 at PRIORITY 0 [ TICKS 4 ALLOT 1 TIME 4 (of 200) ]
[ time 196 ] Run JOB 0 at PRIORITY 0 [ TICKS 3 ALLOT 1 TIME 3 (of 200) ]
[ time 197 ] Run JOB 0 at PRIORITY 0 [ TICKS 2 ALLOT 1 TIME 2 (of 200) ]
[ time 198 ] Run JOB 0 at PRIORITY 0 [ TICKS 1 ALLOT 1 TIME 1 (of 200) ]
[ time 199 ] Run JOB 0 at PRIORITY 0 [ TICKS 0 ALLOT 1 TIME 0 (of 200) ]
[ time 200 ] FINISHED JOB 0

Final statistics:
  Job  0: startTime   0 - response   0 - turnaround 200

  Avg  0: startTime n/a - response 0.00 - turnaround 200.00
```

Example 2:

```
vboxuser@Lucien:~/5600/hw3$ python3 ./mlfq.py -n 3 -q 10 -l 0,180,0:100,20,0 -c
Here is the list of inputs:
OPTIONS jobs 2
OPTIONS queues 3
OPTIONS allotments for queue  2 is   1
OPTIONS quantum length for queue  2 is  10
OPTIONS allotments for queue  1 is   1
OPTIONS quantum length for queue  1 is  10
OPTIONS allotments for queue  0 is   1
OPTIONS quantum length for queue  0 is  10
OPTIONS boost 0
OPTIONS ioTime 5
OPTIONS stayAfterIO False
OPTIONS iobump False


For each job, three defining characteristics are given:
  startTime : at what time does the job enter the system
  runTime   : the total CPU time needed by the job to finish
  ioFreq    : every ioFreq time units, the job issues an I/O
              (the I/O takes ioTime units to complete)

Job List:
  Job  0: startTime   0 - runTime 180 - ioFreq   0
  Job  1: startTime 100 - runTime  20 - ioFreq   0


Execution Trace:

[ time 0 ] JOB BEGINS by JOB 0
[ time 0 ] Run JOB 0 at PRIORITY 2 [ TICKS 9 ALLOT 1 TIME 179 (of 180) ]
[ time 1 ] Run JOB 0 at PRIORITY 2 [ TICKS 8 ALLOT 1 TIME 178 (of 180) ]
[ time 2 ] Run JOB 0 at PRIORITY 2 [ TICKS 7 ALLOT 1 TIME 177 (of 180) ]
```

Example 3:

```
vboxuser@Lucien:~/5600/hw3$ python3 ./mlfq.py -n 3 -q 10 -l 0,175,0:50,25,1 -i 5 -S -c
Here is the list of inputs:
OPTIONS jobs 2
OPTIONS queues 3
OPTIONS allotments for queue  2 is   1
OPTIONS quantum length for queue  2 is  10
OPTIONS allotments for queue  1 is   1
OPTIONS quantum length for queue  1 is  10
OPTIONS allotments for queue  0 is   1
OPTIONS quantum length for queue  0 is  10
OPTIONS boost 0
OPTIONS ioTime 5
OPTIONS stayAfterIO True
OPTIONS iobump False


For each job, three defining characteristics are given:
  startTime : at what time does the job enter the system
  runTime   : the total CPU time needed by the job to finish
  ioFreq    : every ioFreq time units, the job issues an I/O
              (the I/O takes ioTime units to complete)

Job List:
  Job  0: startTime   0 - runTime 175 - ioFreq   0
  Job  1: startTime  50 - runTime  25 - ioFreq   1
```

```
IO DONE
[ time 189 ] Run JOB 0 at PRIORITY 0 [ TICKS 4 ALLOT 1 TIME 9 (of 175) ]
[ time 190 ] Run JOB 0 at PRIORITY 0 [ TICKS 3 ALLOT 1 TIME 8 (of 175) ]
[ time 191 ] Run JOB 0 at PRIORITY 0 [ TICKS 2 ALLOT 1 TIME 7 (of 175) ]
[ time 192 ] Run JOB 0 at PRIORITY 0 [ TICKS 1 ALLOT 1 TIME 6 (of 175) ]
[ time 193 ] Run JOB 0 at PRIORITY 0 [ TICKS 0 ALLOT 1 TIME 5 (of 175) ]
[ time 194 ] IO_DONE by JOB 1
[ time 194 ] Run JOB 1 at PRIORITY 2 [ TICKS 9 ALLOT 1 TIME 0 (of 25) ]
[ time 195 ] FINISHED JOB 1
[ time 195 ] Run JOB 0 at PRIORITY 0 [ TICKS 9 ALLOT 1 TIME 4 (of 175) ]
[ time 196 ] Run JOB 0 at PRIORITY 0 [ TICKS 8 ALLOT 1 TIME 3 (of 175) ]
[ time 197 ] Run JOB 0 at PRIORITY 0 [ TICKS 7 ALLOT 1 TIME 2 (of 175) ]
[ time 198 ] Run JOB 0 at PRIORITY 0 [ TICKS 6 ALLOT 1 TIME 1 (of 175) ]
[ time 199 ] Run JOB 0 at PRIORITY 0 [ TICKS 5 ALLOT 1 TIME 0 (of 175) ]
[ time 200 ] FINISHED JOB 0

Final statistics:
  Job  0: startTime   0 - response   0 - turnaround 200
  Job  1: startTime  50 - response   0 - turnaround 145

  Avg  1: startTime n/a - response 0.00 - turnaround 172.50
```

Q3)
./mlfq.py -n 1 -q 10 -j 2 -c
This makes it so that each job retains the same priority

```
vboxuser@Lucien:~/5600/hw3$ python3 ./mlfq.py -n 1 -q 10 -j 2 -c
Here is the list of inputs:
OPTIONS jobs 2
OPTIONS queues 1
OPTIONS allotments for queue  0 is   1
OPTIONS quantum length for queue  0 is  10
OPTIONS boost 0
OPTIONS ioTime 5
OPTIONS stayAfterIO False
OPTIONS iobump False


For each job, three defining characteristics are given:
  startTime : at what time does the job enter the system
  runTime   : the total CPU time needed by the job to finish
  ioFreq    : every ioFreq time units, the job issues an I/O
              (the I/O takes ioTime units to complete)

Job List:
  Job  0: startTime   0 - runTime  84 - ioFreq   7
  Job  1: startTime   0 - runTime  42 - ioFreq   3
```

Q4)

```
vboxuser@Lucien:~/5600/hw3$ python3 ./mlfq.py -n 2 -q 100 -i 1 -S -l 0,200,0:0,200,99 -c
Here is the list of inputs:
OPTIONS jobs 2
OPTIONS queues 2
OPTIONS allotments for queue  1 is   1
OPTIONS quantum length for queue  1 is 100
OPTIONS allotments for queue  0 is   1
OPTIONS quantum length for queue  0 is 100
OPTIONS boost 0
OPTIONS ioTime 1
OPTIONS stayAfterIO True
OPTIONS iobump False
```

```
Final statistics:
  Job  0: startTime   0 - response   0 - turnaround 400
  Job  1: startTime   0 - response 100 - turnaround 302

  Avg  1: startTime n/a - response 50.00 - turnaround 351.00
```

Q5)

```
vboxuser@Lucien:~/5600/hw3$ python3 ./mlfq.py -B 200 -q 10
Here is the list of inputs:
OPTIONS jobs 3
OPTIONS queues 3
OPTIONS allotments for queue  2 is   1
OPTIONS quantum length for queue  2 is  10
OPTIONS allotments for queue  1 is   1
OPTIONS quantum length for queue  1 is  10
OPTIONS allotments for queue  0 is   1
OPTIONS quantum length for queue  0 is  10
OPTIONS boost 200
OPTIONS ioTime 5
OPTIONS stayAfterIO False
OPTIONS iobump False


For each job, three defining characteristics are given:
  startTime : at what time does the job enter the system
  runTime   : the total CPU time needed by the job to finish
  ioFreq    : every ioFreq time units, the job issues an I/O
              (the I/O takes ioTime units to complete)

Job List:
  Job  0: startTime   0 - runTime  84 - ioFreq   7
  Job  1: startTime   0 - runTime  42 - ioFreq   3
  Job  2: startTime   0 - runTime  51 - ioFreq   4

Compute the execution trace for the given workloads.
If you would like, also compute the response and turnaround
times for each of the jobs.

Use the -c flag to get the exact results when you are finished.
```

Q6)

```
vboxuser@Lucien:~/5600/hw3$ python3 ./mlfq.py -n 3 -q 10 -i 2 -l 0,40,5:0,40,5 -c
Here is the list of inputs:
OPTIONS jobs 2
OPTIONS queues 3
OPTIONS allotments for queue  2 is   1
OPTIONS quantum length for queue  2 is  10
OPTIONS allotments for queue  1 is   1
OPTIONS quantum length for queue  1 is  10
OPTIONS allotments for queue  0 is   1
OPTIONS quantum length for queue  0 is  10
OPTIONS boost 0
OPTIONS ioTime 2
OPTIONS stayAfterIO False
OPTIONS iobump False


For each job, three defining characteristics are given:
  startTime : at what time does the job enter the system
  runTime   : the total CPU time needed by the job to finish
  ioFreq    : every ioFreq time units, the job issues an I/O
              (the I/O takes ioTime units to complete)

Job List:
  Job  0: startTime   0 - runTime  40 - ioFreq   5
  Job  1: startTime   0 - runTime  40 - ioFreq   5


Execution Trace:

[ time 0 ] JOB BEGINS by JOB 0
[ time 0 ] JOB BEGINS by JOB 1
```

```
[ time 77 ] Run JOB 1 at PRIORITY 0 [ TICKS 2 ALLOT 1 TIME 2 (of 40) ]
[ time 78 ] Run JOB 1 at PRIORITY 0 [ TICKS 1 ALLOT 1 TIME 1 (of 40) ]
[ time 79 ] Run JOB 1 at PRIORITY 0 [ TICKS 0 ALLOT 1 TIME 0 (of 40) ]
[ time 80 ] FINISHED JOB 1

Final statistics:
  Job  0: startTime   0 - response   0 - turnaround  75
  Job  1: startTime   0 - response   5 - turnaround  80

  Avg  1: startTime n/a - response 2.50 - turnaround 77.50
```

Chapter 9)
Q1)

```
vboxuser@Lucien:~/5600/hw3$ python3 ./lottery.py -j 3 -s 1 -c
ARG jlist
ARG jobs 3
ARG maxlen 10
ARG maxticket 100
ARG quantum 1
ARG seed 1

Here is the job list, with the run time of each job:
  Job 0 ( length = 1, tickets = 84 )
  Job 1 ( length = 7, tickets = 25 )
  Job 2 ( length = 4, tickets = 44 )


** Solutions **

Random 651593 -> Winning ticket 119 (of 153) -> Run 2
  Jobs:
 (  job:0 timeleft:1 tix:84 )  (  job:1 timeleft:7 tix:25 )  (* job:2 timeleft:4 tix:44 )
```

```
vboxuser@Lucien:~/5600/hw3$ python3 ./lottery.py -j 3 -s 2 -c
ARG jlist
ARG jobs 3
ARG maxlen 10
ARG maxticket 100
ARG quantum 1
ARG seed 2

Here is the job list, with the run time of each job:
  Job 0 ( length = 9, tickets = 94 )
  Job 1 ( length = 8, tickets = 73 )
  Job 2 ( length = 6, tickets = 30 )


** Solutions **

Random 605944 -> Winning ticket 169 (of 197) -> Run 2
  Jobs:
 (  job:0 timeleft:9 tix:94 )  (  job:1 timeleft:8 tix:73 )  (* job:2 timeleft:6 tix:30 )
Random 606802 -> Winning ticket 42 (of 197) -> Run 0
```

```
vboxuser@Lucien:~/5600/hw3$ python3 ./lottery.py -j 3 -s 3 -c
ARG jlist
ARG jobs 3
ARG maxlen 10
ARG maxticket 100
ARG quantum 1
ARG seed 3

Here is the job list, with the run time of each job:
  Job 0 ( length = 2, tickets = 54 )
  Job 1 ( length = 3, tickets = 60 )
  Job 2 ( length = 6, tickets = 6 )


** Solutions **

Random 13168 -> Winning ticket 88 (of 120) -> Run 1
  Jobs:
 (  job:0 timeleft:2 tix:54 )  (* job:1 timeleft:3 tix:60 )  (  job:2 timeleft:6 tix:6 )
Random 837469 -> Winning ticket 109 (of 120) -> Run 1
```

Q2)

```
vboxuser@Lucien:~/5600/hw3$ python3 ./lottery.py -l 10:1,10:100 -c
ARG jlist 10:1,10:100
ARG jobs 3
ARG maxlen 10
ARG maxticket 100
ARG quantum 1
ARG seed 0

Here is the job list, with the run time of each job:
  Job 0 ( length = 10, tickets = 1 )
  Job 1 ( length = 10, tickets = 100 )


** Solutions **

Random 844422 -> Winning ticket 62 (of 101) -> Run 1
  Jobs:
```

Job 0 will only finish first 1% of the time. Imbalances in ticketing will make it as if one job will finish first.

Q3)

```
vboxuser@Lucien:~/5600/hw3$ python3 ./lottery.py -l 100:100,100:100 -c
ARG jlist 100:100,100:100
ARG jobs 3
ARG maxlen 10
ARG maxticket 100
ARG quantum 1
ARG seed 0

Here is the job list, with the run time of each job:
  Job 0 ( length = 100, tickets = 100 )
  Job 1 ( length = 100, tickets = 100 )


** Solutions **

Random 844422 -> Winning ticket 22 (of 200) -> Run 0
  Jobs:
 (* job:0 timeleft:100 tix:100 )  (  job:1 timeleft:100 tix:100 )
Random 757955 -> Winning ticket 155 (of 200) -> Run 1
  Jobs:
 (  job:0 timeleft:99 tix:100 )  (* job:1 timeleft:100 tix:100 )
Random 420572 -> Winning ticket 172 (of 200) -> Run 1
  Jobs:
 (  job:0 timeleft:99 tix:100 )  (* job:1 timeleft:99 tix:100 )
Random 258917 -> Winning ticket 117 (of 200) -> Run 1
  Jobs:
 (  job:0 timeleft:99 tix:100 )  (* job:1 timeleft:98 tix:100 )
Random 511275 -> Winning ticket 75 (of 200) -> Run 0
```

The outcome is probabilistic. From the law of large numbers it should start getting more fair with larger jobs.

Q4) Larger quanta means there are fewer flips. That’s basically the same as decreasing the size of the job. Increases variance.

Q5)
Yes it’s in the github. It’s worth exploring more than two jobs and how it would affect variance. A stride scheduler would make a much flatter curve due to it being more fair.